SymPy Bug Report

1 Bug 47: linsolve misses the a = 0 branch

1.1 Minimal Reproducer

```
from sympy import S, linsolve, symbols

x, a = symbols("x a")
sol = linsolve([a*x - a], [x])
print("solution:", sol)
print("equation at a=0, x=2:", (a*x - a).subs({a: 0, x: 2}))
print("reported contains x=2 after a=0:", (S(2),) in sol.subs(a, 0))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
solution: {(1,)}
equation at a=0, x=2: 0
reported contains x=2 after a=0: False
```

SymPy returns only the singleton solution $\{(1,)\}$. However, after specializing the parameter to $a = 0$, the original equation is satisfied by $x = 2$, while $(2,)$ is not contained in the specialized reported solution. The returned set therefore omits valid solutions on the degenerate parameter branch.

1.3 Expected Behavior

The mathematically correct answer is conditional: $x = 1$ when $a \neq 0$, and all values of x when $a = 0$. Equivalently, the solution set should include the generic branch

$$a \neq 0, \qquad x = 1,$$

and the degenerate branch

$$a = 0, \qquad x \text{ arbitrary.}$$

1.4 Mathematical Explanation

The equation supplied to `linsolve` is

$$ax - a = 0,$$

or, after factoring,

$$a(x - 1) = 0.$$

If $a \neq 0$, division by a is valid and the equation reduces to $x - 1 = 0$, so $x = 1$. If $a = 0$, the equation instead becomes

$$0 \cdot (x - 1) = 0,$$

which is the identity $0 = 0$ and imposes no constraint on x. Thus values such as $x = 2$ are valid solutions on the $a = 0$ branch.

The result $\{(1,)\}$ is not an equivalent representation of the full solution set. Substituting $a = 0$ into the original equation and $x = 2$ gives zero, but substituting $a = 0$ into the reported solution leaves only $\{(1,)\}$, which excludes $(2,)$. The missing branch is therefore a real loss of solutions, not a harmless simplification or alternate set notation.

1.5 Source-Level Diagnosis

The diagnosis identifies the sparse linear-solver domain and row-reduction pipeline as the source-level mechanism. The public call `linsolve([a*x - a], [x])` is handled in `sympy/solvers/solveset.py`, where `linsolve` delegates list inputs to `_linsolve`. That internal solver is imported from `sympy/polys/matrices/linsolve.py`. In the traced path, `_linsolve` converts the equation to coefficient dictionaries, builds a sparse augmented matrix, and computes its reduced row echelon form with `sdm_irref`.

For the representative equation, the linear conversion gives coefficient $\{x : a\}$ and constant $-a$. The matrix-domain construction in `sympy/polys/matrices/linsolve.py` uses the elements of this augmented row to construct the coefficient domain as the rational-function field $\mathbb{Z}(a)$. In that field, a is treated as a nonzero invertible coefficient unless it is the zero rational function. Row reduction therefore divides the row $[a, a]$ by a, obtains the normalized row $[1, 1]$, and returns the generic solution $x = 1$.

```
K, elems_K = construct_domain(elems, field=True, extension=True)
...
Arref, pivots, nzcols = sdm_irref(Aaug)
```

This source-level behavior explains the mathematical failure: the solver has effectively solved the generic branch where $a \neq 0$, but the returned `FiniteSet` carries no condition $a \neq 0$ and does not split off the specialization $a = 0$. The diagnosis presents a plausible fix direction: for systems with symbolic coefficients, `linsolve` should either document and return only generic solutions, or carry pivot nonzero conditions and split degenerate parameter branches. In this case, branch-aware handling would return $x = 1$ for $a \neq 0$ and all x for $a = 0$.

1.6 Additional Failing Instances

The independent verification checks the family

$$a(x - r) = 0$$

for integer roots $r = 1, 2, 3, 4, 5, 6$. In each case, the generic solution is $x = r$, but the specialization $a = 0$ turns the equation into $0 = 0$, so every x is a solution. The verification chooses $x = r + 1$ as an explicit omitted solution on the $a = 0$ branch.

Input / Expression	SymPy behavior	Expected behavior
linsolve($[a(x-1)], [x]$)	Returns $\{(1,)\}$; after $a=0$, omits $(2,)$	$x=1$ if $a\neq 0$; all x if $a=0$
linsolve($[a(x-2)], [x]$)	Returns $\{(2,)\}$; after $a=0$, omits $(3,)$	$x=2$ if $a\neq 0$; all x if $a=0$
linsolve($[a(x-3)], [x]$)	Returns $\{(3,)\}$; after $a=0$, omits $(4,)$	$x=3$ if $a\neq 0$; all x if $a=0$
linsolve($[a(x-4)], [x]$)	Returns $\{(4,)\}$; after $a=0$, omits $(5,)$	$x=4$ if $a\neq 0$; all x if $a=0$
linsolve($[a(x-5)], [x]$)	Returns $\{(5,)\}$; after $a=0$, omits $(6,)$	$x=5$ if $a\neq 0$; all x if $a=0$
linsolve($[a(x-6)], [x]$)	Returns $\{(6,)\}$; after $a=0$, omits $(7,)$	$x=6$ if $a\neq 0$; all x if $a=0$

The same reasoning applies uniformly across the family. The factor a is the only coefficient carrying the parameter branch, so specializing $a = 0$ annihilates the equation regardless of the integer root r.

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. This specific failure mode is documented in the open issue sympy/sympy#16861, "Proper solution of linear equations with symbolic coefficients" (open, 18 comments, opened 2019). That issue raises the same symbolic-coefficient missing-$a = 0$-branch problem using the essentially identical example $ax = b$ (this bug's $ax - a$ is the $b = a$ case), observes that the solve/solveset/linsolve solutions "are not valid for a=0", and proposes a conditional `linsolve` variant that carries the degenerate $a = 0$ branch. Consistent with this, the SymPy documentation shows `linsolve` solving systems with symbolic coefficients by producing generic determinant-denominator formulas without splitting singular-parameter cases, which is its long-standing documented generic-coefficient convention. The deduplication verdict is: family known, specific instance. The exact reproducer `linsolve([a*x - a], [x])` and the result $\{(1,)\}$ fall squarely within the failure mode #16861 describes and remain individually actionable as a regression test, but this is a known, long-standing limitation rather than a previously unreported failure mode.

1.8 Proposed SymPy Regression Test

```python
import pytest
from sympy import S, linsolve, symbols

x, a = symbols("x a")

@pytest.mark.parametrize("root", [1, 2, 3, 4, 5, 6])
def test_linsolve_keeps_zero_parameter_branch(root):
    expr = a*(x - root)
    sol = linsolve([expr], [x])
    extra_solution = S(root + 1)
```

```
    assert (extra_solution,) in sol.subs(a, 0)
```